# JavaScript Cheatsheet (Chapter 4)

Oct 19, 2026 · @lou

## 1. Adding JS to a page

HTML/CSS are declarative; JS adds behavior. The browser runs scripts, top to bottom, with no `main()`.

| Method | Syntax | Use |
|---|---|---|
| Inline | `<button onclick="alert('Hi')">` | Tiny one-liners only (discouraged) |
| Script block | `<script> ... </script>` | Small pages; put it at the end of `<body>` |
| External file | `<script src="mon_script.js" defer></script>` | Default choice; can load several files or a CDN URL |

**Classic beginner error:** a script in `<head>` runs before the DOM exists, so `getElementById` returns `null`. Fix it with `defer`, a script at the end of `<body>`, or `document.addEventListener('DOMContentLoaded', ...)`.

## 2. Language basics

JS is interpreted (JIT in the browser), dynamically and weakly typed, single-threaded with an event loop, and garbage-collected. It is unrelated to Java.

```
console.log("Salut", x);   // debug output: DevTools > Console
alert("Hello");             // popup
elt.innerHTML = "Hello";    // write into the page

let age = 30;               // number (int or float)
let nom = 'JOHN';           // string; backticks allow `Bonjour ${nom}`
let ok = true;              // boolean
let x = null;               // empty value
const pi = 3.14;            // constant
2 + "2"                     // "22" (automatic type conversion)
```

### Arrays

```
const a = [1, 2, 3, 4, 5];
a[0];  a.push(6);  a.pop();  a.includes(2);       // access, add, remove
last, test
a.find(n => n > 3);                               // 4 (first match)
a.filter(n => n > 3);                             // [4, 5]
a.map(n => n * 2);                                // [2, 4, 6, 8, 10]
a.reduce((acc, val) => acc + val, 0);             // 15
const [x, y, z] = a;                              // destructuring: 1, 2, 3
```

**Objects** (almost everything in JS is one)

```
const person = { nom: 'JOHN', age: 30, adresse: { ville: 'Alger' } };
person.nom;  person["age"];        // access
person.metier = "Ingenieur";       // add or modify
delete person.metier;              // remove
const { nom, age } = person;       // destructuring
JSON.stringify(etudiants);         // array/object to JSON string (client-
server exchange)
```

## 3. Operators, conditions, loops

Syntax is close to C: `;` ends a statement (optional), `{ }` makes a block, `//` and `/* */` are comments.

| Kind | Operators |
|---|---|
| Arithmetic | `+ - * / % ** ++ --` |
| Combined | `+= -= *= /= %= **=` |
| Logical | `&&` (and), `\|\|` (or), `!` (not) |
| Comparison | `> < >= <= == !=` |
| Strict equality | `5 == "5"` is true, `5 === "5"` is false. Prefer `===` |

```
if (a > b) { ... } else if (a < b) { ... } else { ... }
const res = (val === 1) ? true : false;     // ternary

switch (jour) { case 1: nom = "Lundi"; break; default: nom = "Inconnu"; }

for (let i = 0; i < arr.length; i++) { ... }
while (i < arr.length) { ...; i++; }
for (const e of arr) { ... }                // values
arr.forEach(e => { ... });                  // no break, continue or return
```

## 4. Functions and async

| Type | Example | Hoisted | `this` |
|---|---|---|---|
| Declaration | `function f(val=1) { return val*3; }` | Yes | Its own |
| Arrow | `const f = (val=1) => val*3;` | No | Parent's (good for callbacks) |
| IIFE (anonymous, runs at once) | `(function() { ... })();` | No | Its own (protects the code, not reusable) |

Objects and arrays are passed by reference, so a function that changes `obj.id` changes the original.

**Callback:** a function passed to another function and called later.

```
function processData(data, callback) { callback(data); }
processData("Bonjour", (result) => console.log(result));
```

**Promise:** an object standing for a result (or error) that arrives later.

```
const p = new Promise((resolve, reject) => setTimeout(() => resolve("Done"),
1000));
p.then(r => console.log(r)).catch(e => console.error(e));
console.log("runs while waiting");   // not blocked
```

**async/await:** the same thing, easier to read.

```
async function fetchData() {
  try { const r = await f(); console.log(r); }
  catch (e) { console.error(e); }
}
```

**Event loop:** the call stack runs your code. Async work (`setTimeout`, AJAX, DOM events) goes to the browser's Web APIs, and each finished callback waits in the callback queue until the stack is empty.

## 5. Scope

Always declare variables: `let` for variables, `const` for constants. Skipping the keyword creates an implicit global, even inside a block or function.

- `let` / `const`: limited to the current block (`if`, loop) or function.
- `var`: old style; hoisted, so it leaks out of blocks. Avoid it.
- Add `"use strict";` at the top so undeclared variables throw a `ReferenceError`.

## 6. Events and DOM

Every interaction follows the same pattern: an event fires, a handler runs, and the page visibly changes.

```
<h1 id="compt_H1">Compteur=0</h1>
<button onclick="doIncrement()">Incrémenter</button>
<script>
  let compt = 0;
  function doIncrement() {
    compt++;
    document.getElementById('compt_H1').innerHTML = "Compteur=" + compt;   //
change content
    if (compt > 5) document.getElementById('compt_H1').style.color = "red";
// change style
  }
</script>
```

| Step | What it is | Examples |
| --- | --- | --- |
| Event | Something happens | `onclick`, `onmouseover`, `onload`, `DOMContentLoaded` |
| Processing | JS logic | Variables, conditions, loops, functions |
| Visual change | Select an element, then change it | `.innerHTML`, `.innerText`, `.style.color` |